

© Akshat Paul and Abhishek Nalwaya 2019

Akshat Paul and Abhishek Nalwaya, *React Native for Mobile Development*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-4454-8_8

8. Testing

Akshat Paul¹ and Abhishek Nalwaya²

(1) Gurgaon, Haryana, India

(2) Jaipur, Rajasthan, India

Testing is not the point. The point is about responsibility.

—Kent Beck

We do agree with Kent Beck here that testing your code is your responsibility. However, React Native makes it really simple to write tests for your application in comparison to traditional native code for iOS and Android. This chapter covers the following topics:

- Static type checking with Flow
- Jest with React Native
- Snapshot testing

Flow

Flow is a static type checker for JavaScript. It's not essential to use Flow, but it really enhances your development efficiency. Type checking allows you to detect possible issues early by running tests on your project code base. In short, we would say Flow is a productivity module for developers.

To set up Flow for React Native applications, first open your terminal and simply install the following `npm` module:

```
$ npm install -g flow-bin
```

This will install the `flow` module globally. Navigate to your React Native application folder and from the root directory, execute the following command:

```
$ flow init
```

This will create a `.flowconfig` file where all your Flow configurations will reside.

Now, check your application for any errors with the following command:

```
$ flow check  
Found 0 errors
```

Typically, at the beginning of any project, you will find no errors. As you proceed with day-to-day development, however, you can find issues right away and resolve them.

Benefits of Using Flow

Although Flow is a great addition to any React Native application, it's not mandatory. However, we recommend that you include it in your React Native project to experience the following benefits:

- You can code faster without the hassle of running the source code every time to find any issues or bugs.
- It is especially helpful for extensive projects with multiple team members working in parallel. Refactoring can become a nightmare, and Flow helps you focus only on your changes and eliminates worry about breaking other parts of the source code.
- Flow helps developers to understand idiomatic JavaScript. It understands and provides feedback on common JavaScript patterns, which helps developers to create elegant solutions.
- Flow provides real-time feedback, hence saving a great deal of time and improving code quality.
- Flow provides easy integration. As seen earlier, it takes only a few minutes to set up Flow with your project.

Jest

Jest is a unit test framework that is built on top of Jasmine. React Native supports testing of components using Jest (it's also the recommended framework used at Facebook for React Native). Besides React Native, you can also use Jest for other JavaScript projects built using TypeScript, Node, Angular, React for Web, Vue, and many more.

Key features of the Jest testing framework include the following:

- *Snapshot testing:* Jest allows you to create tests that keep track of large objects. This helps you to write better test cases of UI elements.
- *Zero configuration:* Jest works out of the box and is configuration free.
- *Fast and isolated:* Tests are parallelized by running them in their own processes, which helps maximize performance. Jest runs previous failed tests first and reorganizes the runs based on how long it took to execute the tests.
- *Simple APIs:* Jest makes use of simple conventions that developers are used to. Jest covers the entire toolkit, with updated documentation that is well maintained.
- *Code coverage:* No additional setup is required to pull a built-in code coverage report.

Jest with React Native

Jest is included out of the box with the React Native framework for versions 0.38 and later. You are not required to use Jest, though. Instead, you can also use a Mocha testing framework. When you set up the project initially, create a new project with the following command:

```
$ react-native init jestBasics
```

You will get Jest preloaded, and the following `package.json` code will already be present:

```
"scripts": {  
  ...  
  "test": "jest"  
  ...  
},  
"devDependencies": {  
  ...  
  "jest": "24.1.0",  
"react-test-renderer": "16.6.3"  
  ...  
},  
"jest": {  
  ...  
  "preset": "react-native"  
  ...  
}
```

There will be a folder created, `__tests__`, which includes only one file for now, `App.js`:

```
/**
 * @format
 * @lint-ignore-every XPLATJSCOPYRIGHT1
 */

import 'react-native';
import React from 'react';
import App from '../App';
// Note: test renderer must be required after react-native.
import renderer from 'react-test-renderer';
it('renders correctly', () => {
  renderer.create(<App />);
});
```

If you run `yarn test` or `npm test`, your tests will run. Because is there nothing much there yet, you should get the following result:

```
$ yarn test
yarn run v1.9.4
$ jest
PASS  __tests__/App.js
  ✓ renders correctly (2650ms)
Test Suites: 1 passed, 1 total
Tests:      1 passed, 1 total
```

```
Snapshots:    0 total
Time:         5.756s
Ran all test suites.
```

This was the case if you generated your project using the React Native CLI, but what about Expo? In previous chapters we used Expo to speed up our development process. With Expo the setup process is slightly different because the Expo CLI does not come with Jest out of the box.

For this you would have to set up Jest manually with an `include jest-expo`, which is not very complicated. You could try this in an existing Expo application you created in a previous chapter or set up a new one.

Navigate to your project folder and add `jest-expo` to your project using the following command:

```
$ yarn add jest-expo --dev or $ npm i jest-expo --save-dev
```

Open `package.json` and add the following code:

```
"scripts": {
  "test": "node_modules/.bin/jest"
},
"jest": {
  "preset": "jest-expo"
```

```
}
```

Also, create a `__test__` folder and add at least one test file with the following sample test:

```
it('works', () => {  
  expect(1).toBe(1);  
});
```

Open the terminal and run the following code:

```
$ yarn test OR npm test  
yarn test  
yarn run v1.9.4  
$ node_modules/.bin/jest  
PASS  __tests__/Example-test.js  
  ✓ works (3ms)  
Test Suites: 1 passed, 1 total  
Tests:      1 passed, 1 total
```

```
Snapshots: 0 total  
Time:      2.288s  
Ran all test suites.  
Done in 3.75s.
```

Snapshot Testing with Jest

Snapshotting is a really useful technique in UI development that helps ensure that there are no unexpected changes in the UI during development. With Jest we can capture snapshots of React trees, which help us to compare if there was a breaking change in subsequent changes.

A snapshot test case for a mobile app will render a UI component, take a snapshot, and then compare it to a reference point in the past by storing a snapshot alongside the test case. If the test fails, that means two snapshots did not match due to an unexpected change in the UI. Snapshots should be updated to a new version when a satisfactory UI component is ready.

Let's add our first snapshot test and check the report. Add the following test in the same `App.js` file. For ease of understanding we are using a project created with the React Native CLI named `jestBasics`.

```
test("component renders correctly", () => {  
  const tree = renderer.create(<App />).toJSON();  
  expect(tree).toMatchSnapshot();  
});
```

Run the Jest report again with the following command:

```
npm test  
> jestBasics@0.0.1 test /Users/akshatpaul/myapps/react-native-second-edition/chapter8/jestBasics
```

```
> jest
PASS  __tests__/App.js
  ✓ renders correctly (132ms)
  ✓ component renders correctly (5ms)
  › 1 snapshot written.
Snapshot Summary
  › 1 snapshot written from 1 test suite.
Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
```

Snapshots: 1 written, 1 total

```
Time:        1.114s, estimated 2s
Ran all test suites.
```

Great! Our test passed and created a snapshot of the render output of our component. This snapshot is saved in a new folder, `__snapshots__`, which resides inside the `__test__` folder. You will find a snapshot file `App.js.snap`. Open that file if you want to see what's inside a snapshot.

```
// Jest Snapshot v1, https://goo.gl/fbAQLP
exports['component renders correctly 1'] = '
<View
  style={
    Object {
      "alignItems": "center",
      "backgroundColor": "#F5FCFF",
      "flex": 1,
      "justifyContent": "center",
```

```
    }  
  }  
>  
<Text  
  style={  
    Object {  
      "fontSize": 10,  
      "margin": 10,  
      "textAlign": "center",  
    }  
  }  
>  
  Welcome to React Native!  
</Text>  
<Text  
  style={  
    Object {  
      "color": "#333333",  
      "marginBottom": 5,  
      "textAlign": "center",  
    }  
  }  
>  
  To get started, edit App.js  
</Text>  
<Text  
  style={  
    Object {  
      "color": "#333333",  
      "marginBottom": 5,
```

```
      "textAlign": "center",
    }
  }
}
>
  Press Cmd+R to reload,
Cmd+D or shake for dev menu
</Text>
</View>
`;
```

Do not modify this snapshot. Instead, make some change in your `App.js` component and see how the snapshot changes and the report fails. Let's make the following change in our styling:

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'flex-start',
    backgroundColor: '#F5FCFF',
  },
  welcome: {
    fontSize: 10,
    textAlign: 'center',
    margin: 10,
  },
  instructions: {
    textAlign: 'center',
    color: '#333333',
    marginBottom: 5,
  },
});
```

```
    },
  });

```

Here we have only made one small change in `alignItems` ,
changing the setting from `center` to `flex-start`. Run the report
again and see if the test fails:

```
npm test
> jestBasics@0.0.1 test /Users/akshatpaul/myapps/react-native-second-edition/chapter8/jestBasics
> jest
FAIL  __tests__/App.js
  ✓ renders correctly (135ms)
  × component renders correctly (9ms)
    • component renders correctly
      expect(value).toMatchSnapshot()
      Received value does not match stored snapshot "component renders correctly 1".
# - Snapshot
+ Received
@@ -1,9 +1,9 @@
  <View
    style={
      Object {
-       "alignItems": "center",
+       "alignItems": "flex-start",
        "backgroundColor": "#F5FCFF",
        "flex": 1,
        "justifyContent": "center",
      }
    }
  18 |   const tree = renderer.create(<App />).toJSON();

```

```
19 |  
> 20 |   expect(tree).toMatchSnapshot();  
    |           ^  
21 | });  
22 |  
    at Object.toMatchSnapshot (__tests__/App.js:20:16)  
> 1 snapshot failed.  
Snapshot Summary  
  > 1 snapshot failed from 1 test suite. Inspect your code changes or run 'npm test -- -u' to update them.  
Test Suites: 1 failed, 1 total  
Tests:       1 failed, 1 passed, 2 total  
Snapshots:   1 failed, 1 total  
Time:        0.886s, estimated 1s  
Ran all test suites.  
npm ERR! Test failed.  See above for more details.
```

Perfect! Our test failed, and this shows how snapshot testing with Jest really helps during development of a substantial React Native application if one developer makes a change, for example, that might hinder the UI build by someone else.

Summary

Testing is a crucial component in any mobile app development. In this chapter you learned about using Flow to keep your code type checked to assist in detecting issues with your code early and resolving them before they become bugs. Next, you learned about testing with Jest and how to set it up for both React Native CLI apps and those generated using the ExpoCLI.

In the end, we introduced the powerful technique of snapshot testing with Jest, which makes building UIs and maintaining them much easier.

Chapter **9** covers iOS and Android app submission to the Apple App Store and Google Play Store, respectively.